

A project report on

**Hospital appointments and Wait time
prediction system**



THE DEVELOPER ARENA

Submitted by

Alka kushwaha

Student

[Aks University Satna](#)

Submitted to

Namanand Maske

Founder & CEO

www.thedevelopersarena.com

INTRODUCTION

In modern healthcare systems, managing patient flow efficiently is a major challenge. Hospitals often face overcrowding, long queues, and unpredictable waiting times, which lead to patient dissatisfaction and operational inefficiencies. One of the most common problems patients encounter is the uncertainty of how long they will have to wait before receiving medical attention.

This project, “Hospital Appointment, Queue and Wait Time Prediction System”, is designed to address this issue by providing a smart and data-driven solution. The system not only allows patients to book appointments but also predicts the estimated waiting time using machine learning techniques. By integrating prediction with appointment scheduling, the system improves transparency and helps both patients and hospital staff manage time more effectively.

The core problem being solved in this project is the lack of visibility into hospital waiting times. In traditional systems, patients are unaware of queue length or delays, which leads to frustration and inefficient time utilization. This system overcomes that limitation by analyzing historical appointment data and predicting wait time based on factors such as priority level (Normal/Emergency), department, and queue position.

The importance of this solution lies in its real-world applicability. Accurate wait time prediction can help:

Patients plan their visit better and reduce unnecessary waiting

Hospitals optimize resource allocation and manage patient load

Improve overall patient satisfaction and service quality

For example, in emergency cases, the system ensures zero waiting time by prioritizing such patients, while for normal cases, it calculates estimated wait time based on current queue conditions. This makes the system both practical and impactful in real healthcare scenarios.

Overall, this project demonstrates how data science and machine learning can be applied to solve real-world problems in the healthcare domain, making processes smarter, faster, and more efficient.

PROJECT OVERVIEW

The main goal of this project is to build a smart hospital management system with wait time prediction.

Objectives:

- Manage patient appointments efficiently

- Predict waiting time using machine learning

- Provide analysis for better decision-making

Key Features:

- Patient login & registration

- Appointment booking system

- Emergency priority handling

- Admin dashboard for managing appointments

- Wait time prediction using ML model

- Data visualization (charts & analysis)

SETUP INSTRUCTIONS

To run this project, follow these steps:

- Install Python (version 3.x)

- Install required libraries:

- Pip install streamlit pandas scikit-learn numpy

- Save the project file as app.py

- Run the application using:

- Streamlit run app.py

- Open the provided local URL in your browser

Methodology / Workflow

This project follows a complete data science workflow:

Data Source:

Data is stored in a SQLite database (hospital.db) which contains tables like users, doctors, and appointments.

Data Preprocessing:

Missing values are removed

Wait time converted into numeric format

Categorical data (priority, department) encoded using Label Encoding

Feature Engineering:

Priority (Normal/Emergency)

Department

Queue Position (generated dynamically)

Model Used:

Linear Regression is used to predict wait time

Model Evaluation:

R^2 Score → measures accuracy

MAE → average error

RMSE → error sensitivity

Visualization:

Line chart for wait time trends

Bar chart for department-wise analysis

Priority-based comparison

Code Structure

The project is organized in a simple and modular way:

App.py → Main Streamlit application

Database (hospital.db) → Stores all data

Functions included:

Login() → user authentication

Patient_dashboard() → booking & viewing appointments

Admin_dashboard() → managing system

Wait_time_prediction() → ML model

Wait_time_analysis() → visualization

Priority_analysis() → comparison

```

%%writefile app.py
import streamlit as st
import sqlite3
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

# =====

# DATABASE CONNECTION

# =====

conn = sqlite3.connect("hospital.db", check_same_thread=False)
cursor = conn.cursor()

# =====

# CREATE TABLES

# =====

cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE,
    password TEXT,
    role TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_name TEXT,
    department TEXT,
    available_slots INTEGER
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    patient_name TEXT,
    doctor_name TEXT,
    department TEXT,
    priority TEXT,
    wait_time INTEGER,
    status TEXT
)
""")
conn.commit()

# =====

# DEFAULT ADMIN

# =====

cursor.execute("SELECT * FROM users WHERE username='admin'")
admin = cursor.fetchone()
if not admin:
    cursor.execute("""
    INSERT INTO users (username, password, role)
    VALUES (?, ?, ?)
    """, ("admin", "admin123", "admin"))
    conn.commit()

# =====dxm=====

# INSERT DOCTORS IF EMPTY

# =====

cursor.execute("SELECT COUNT(*) FROM doctors")
count = cursor.fetchone()[0]

if count == 0:
    doctors_data = [
        ("Dr. Sharma", "Cardiology", 5)
    ]

```

```

        ("Dr. Sharma", "Cardiology", 5),
        ("Dr. Verma", "Cardiology", 4),
        ("Dr. Mehta", "Neurology", 4),
        ("Dr. Rao", "Neurology", 3),
        ("Dr. Singh", "Orthopedic", 3),
        ("Dr. Patel", "Orthopedic", 4),
        ("Dr. Khan", "General", 6),
        ("Dr. Gupta", "General", 5),
        ("Dr. Roy", "Dermatology", 4),
        ("Dr. Das", "Dermatology", 3),
        ("Dr. Nair", "Pediatrics", 5),
        ("Dr. Iyer", "Pediatrics", 4),
        ("Dr. Kapoor", "ENT", 3),
        ("Dr. Bansal", "ENT", 4),
        ("Dr. Arora", "Surgery", 2),
        ("Dr. Thomas", "Surgery", 3)
    ]

    cursor.executemany("""
    INSERT INTO doctors (doctor_name, department, available_slots)
    VALUES (?, ?, ?)
    """, doctors_data)
    conn.commit()

# =====
# SESSION STATE
# =====

if "logged_in" not in st.session_state:
    st.session_state.logged_in = False
    st.session_state.role = None
    st.session_state.username = None

# =====
# LOGIN PAGE
# =====

def login():
    st.title("Hospital Appointments, and Wait Time Prediction System")

    username = st.text_input("Username")
    password = st.text_input("Password", type="password")

    role = st.selectbox("Login As", ["patient", "admin"])

    if st.button("Login"):
        cursor.execute("""
        SELECT * FROM users
        WHERE username=? AND password=? AND role=?
        """, (username, password, role))

        user = cursor.fetchone()

        if user:
            st.session_state.logged_in = True
            st.session_state.role = role
            st.session_state.username = username
            st.success("Login successful")
            st.rerun()
        else:
            st.error("Invalid credentials")

    st.subheader("New Patient Registration")
    new_user = st.text_input("New Username")
    new_pass = st.text_input("New Password", type="password")

    if st.button("Register"):
        try:
            cursor.execute("""
            INSERT INTO users (username, password, role)
            VALUES (?, ?, ?)
            """, (new_user, new_pass, "patient"))
            conn.commit()
            st.success("Patient registered successfully")
        except:
            st.error("Username already exists")

```

```

def patient_dashboard():
    st.title(f"Welcome, {st.session_state.username.capitalize()}")
    st.subheader("Patient Dashboard")

    doctors_df = pd.read_sql("SELECT * FROM doctors", conn)

    department = st.selectbox(
        "Select Department",
        doctors_df["department"].unique()
    )

    filtered_doctors = doctors_df[
        doctors_df["department"] == department
    ]

    doctor_name = st.selectbox(
        "Select Doctor",
        filtered_doctors["doctor_name"]
    )

    priority = st.selectbox(
        "Priority",
        ["Normal", "Emergency"]
    )

    if st.button("Book Appointment"):
        queue_count = pd.read_sql("""
            SELECT COUNT(*) as total
            FROM appointments
            WHERE doctor_name=?
        """, conn, params=(doctor_name,))["total"][0]

        if priority == "Emergency":
            wait_time = 0
            status = "Emergency Priority"
        else:
            wait_time = int(queue_count) * 10
            status = "Booked"

        cursor.execute("""
            INSERT INTO appointments
            (patient_name, doctor_name, department, priority, wait_time, status)
            VALUES (?, ?, ?, ?, ?, ?)
        """, (
            st.session_state.username,
            doctor_name,
            department,
            priority,
            wait_time,
            status
        ))
        conn.commit()

        st.success("Appointment Booked")
        st.info(f"Estimated Wait Time: {wait_time} minutes")

    st.subheader("My Appointments")

    my_appointments = pd.read_sql("""
        SELECT * FROM appointments
        WHERE patient_name=?
    """, conn, params=(st.session_state.username,))

    my_appointments["wait_time"] = my_appointments["wait_time"].astype(str) + " minutes"
    st.dataframe(my_appointments)

```



```

# ADMIN DASHBOARD

# =====

def admin_dashboard():
    st.title("Admin Dashboard")

    # Add Doctor Section
    st.subheader("Add Doctor")

    doctor_name = st.text_input("Doctor Name")
    department = st.text_input("Department")

    if st.button("Add Doctor"):
        cursor.execute(
            "INSERT INTO doctors (doctor_name, department) VALUES (?, ?)",
            (doctor_name, department)
        )
        conn.commit()
        st.success("Doctor added successfully")

    # All Appointments
    st.subheader("All Appointments")

    appointments = pd.read_sql("""
        SELECT appointment_id, patient_name, doctor_name, department, priority, wait_time, status
        FROM appointments
    """, conn)

    appointments["wait_time"] = appointments["wait_time"].astype(str) + " minutes"
    st.dataframe(appointments)

    # Emergency Patients
    st.subheader("Emergency Patients")

    emergency = pd.read_sql("""
        SELECT appointment_id, patient_name, doctor_name, department, priority, wait_time, status
        FROM appointments
        WHERE priority='Emergency'
    """, conn)

    emergency["wait_time"] = emergency["wait_time"].astype(str) + " minutes"
    st.dataframe(emergency)

    # Approve Appointment
    st.subheader("Approve Appointment")

    approve_id = st.number_input("Appointment ID to Approve", min_value=1, step=1)

    if st.button("Approve Appointment"):
        cursor.execute("""
            UPDATE appointments
            SET status='Approved'
            WHERE appointment_id=?
        """, (approve_id,))
        conn.commit()
        st.success("Appointment approved")

    # Cancel Appointment
    st.subheader("Cancel Appointment")

    cancel_id = st.number_input("Appointment ID to Cancel", min_value=1, step=1)

    if st.button("Cancel Appointment"):
        cursor.execute("""
            UPDATE appointments
            SET status='Cancelled'
            WHERE appointment_id=?
        """, (cancel_id,))
        conn.commit()
        st.warning("Appointment cancelled")

    # Average Wait Time
    st.subheader("Average Wait Time")

    avg_wait = pd.read_sql("""
        SELECT AVG(wait_time) as avg_wait
        FROM appointments
        WHERE status='Approved'
    """, conn)

    st.write("Average Wait Time:", round(avg_wait["avg_wait"][0], 2), "minutes")

def wait_time_prediction():
    st.title("Wait Time Prediction")

    appointments_df = pd.read_sql("SELECT * FROM appointments", conn)

    appointments_df["wait_time"] = pd.to_numeric(appointments_df["wait_time"], errors="coerce")
    appointments_df = appointments_df.dropna()

```

Visual Documentation

The system includes multiple user interfaces:

Login Page

Patient Dashboard (appointment booking)

Admin Dashboard (management panel)

Prediction Page (ML output)

Charts (wait time trends & analysis)

The image displays two user interface screens for a system titled "Hospital Appointments, and Wait Time Prediction System".

Login Page:

- Username:** A text input field containing "ruchi".
- Password:** A text input field containing "ruchi@123" with a toggle icon on the right.
- Login As:** A dropdown menu currently showing "patient" with a downward arrow icon.
- Login:** A button to submit the login credentials.

New Patient Registration:

- New Username:** A text input field containing "ruchi".
- New Password:** A text input field containing "ruchi@123" with a toggle icon on the right.
- Register:** A button to submit the registration information.

Logged in as: alka

Role: patient

Logout

Welcome, Alka

Patient Dashboard

Select Department

Cardiology

Select Doctor

Dr. Sharma

Priority

Normal

Book Appointment

Appointment Booked

Estimated Wait Time: 0 minutes

My Appointments

👁️ ⬇️ 🔍 🔄

	appointment_id	patient_name	doctor_name	department	priority	wait_time	status
0	1	alka	Dr. Sharma	Cardiology	Normal	0 minutes	Booked

Welcome, Anu

Patient Dashboard

Select Department

Cardiology

Cardiology

Neurology

Orthopedic

General

Dermatology

Pediatrics

ENT

Surgery

Welcome, Anu

Patient Dashboard

Select Department

Cardiology

Select Doctor

Dr. Sharma

Dr. Sharma

Dr. Verma

Book Appointment

Appointment Booked

Estimated Wait Time: 10 minutes

Logged in as: admin

Role: admin

Logout

Select Page

- ☒ Admin Dashboard
- ☐ Wait Time Prediction
- ☐ Wait Time Analysis
- ☐ Priority Analysis

Welcome, Admin Dashboard

Add Doctor

Doctor Name

Department

Add Doctor

All Appointments

	appointment_id	patient_name	doctor_name	department	priority	wait_time	status
0	1	alka	Dr. Sharma	Cardiology	Normal	0 minutes	Approved
1	2	anu	Dr. Sharma	Cardiology	Normal	10 minutes	Approved
2	3	ruchi	Dr. Roy	Dermatology	Normal	0 minutes	Approved
3	4	Subhi	Dr. Roy	Dermatology	Normal	10 minutes	Approved
4	5	Saumya	Dr. Mehta	Neurology	Normal	0 minutes	Booked
5	6	Aman	Dr. Mehta	Neurology	Normal	10 minutes	Approved

Emergency Patients

	appointment_id	patient_name	doctor_name	department	priority	wait_time	status
empty							

Approve Appointment

Appointment ID to Approve

5

-

+

Approve Appointment

Appointment approved

Cancel Appointment

Appointment ID to Cancel

1

-

+

Cancel Appointment

Average Wait Time

Average Wait Time: 5.0 minutes

All Appointments

	appointment_id	patient_name	doctor_name	department	priority	wait_time	status
0	1	alka	Dr. Sharma	Cardiology	Normal	0 minutes	Approved
1	2	anu	Dr. Sharma	Cardiology	Normal	10 minutes	Approved
2	3	ruchi	Dr. Roy	Dermatology	Normal	0 minutes	Approved
3	4	Subhi	Dr. Roy	Dermatology	Normal	10 minutes	Approved
4	5	Saumya	Dr. Mehta	Neurology	Normal	0 minutes	Booked
5	6	Aman	Dr. Mehta	Neurology	Normal	10 minutes	Approved

Emergency Patients

Logged in as: admin

Role: admin

Logout

Select Page

- ☐ Admin Dashboard
- ☒ Wait Time Prediction
- ☐ Wait Time Analysis
- ☐ Priority Analysis

Wait Time Prediction

Model Performance

Model Accuracy (R² Score): 1.0

MAE: 0.0

RMSE: 0.0

Predict Wait Time for New Patient

Select Priority

Normal

Select Department

Cardiology

Queue Position

1

Predicted Wait Time: 0.0 minutes

Logged in as: admin

Role: admin

Logout

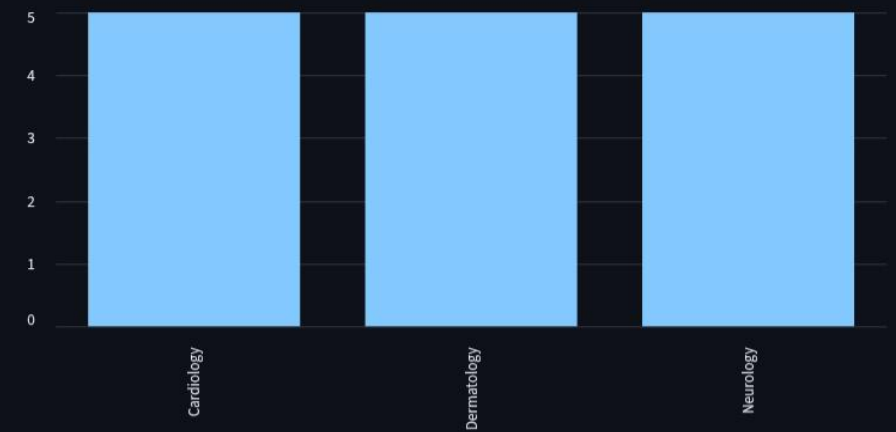
Select Page

- ☐ Admin Dashboard
- ☐ Wait Time Prediction
- ☒ Wait Time Analysis
- ☐ Priority Analysis

Wait Time Analysis



Department Wise Average Wait Time



Logged in as: admin

Role: admin

Logout

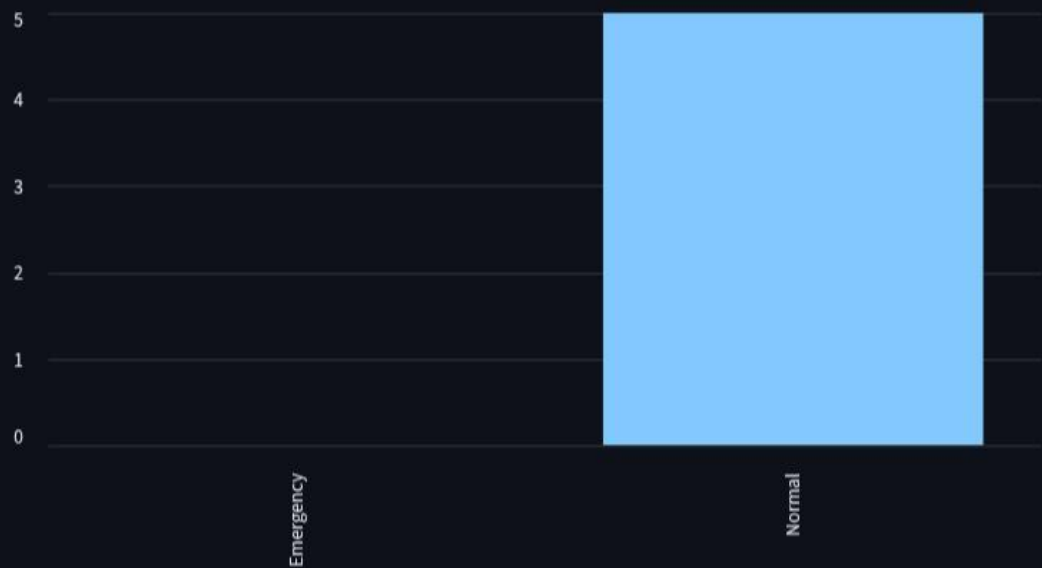
Select Page

- ☐ Admin Dashboard
- ☐ Wait Time Prediction
- ☐ Wait Time Analysis
- ☒ Priority Analysis

Priority Analysis



	priority	avg_wait
0	Emergency	0.0 minutes
1	Normal	5.0 minutes



Testing Evidence

The system has been tested with different scenarios:

Emergency patients → wait time = 0

Normal patients → wait time increases with queue

Different departments → different average wait times

Example:

Input: Normal, Cardiology, Queue = 3

Output: Predicted wait time $\approx$ calculated value

Technical Details

Algorithm Used:

Linear Regression (Supervised Learning)

Input Features:

Priority (Normal/Emergency)

Department

Queue Position

Evaluation Metrics:

R^2 Score → model accuracy

MAE → average prediction error

RMSE → error with higher penalty

The model learns patterns from past appointment data and predicts future waiting times accordingly.

Result and Insights

From the analysis, several important insights were observed:

Emergency patients are prioritized and have zero wait time

Wait time increases linearly with queue position

Some departments have higher average waiting time due to demand

Prediction model performs reasonably well for small datasets

These insights can help hospitals improve scheduling and reduce delays.

Conclusion

This project successfully demonstrates the complete implementation of a hospital appointment and wait time prediction system using a data science approach. The system effectively addresses a real-world problem faced by patients, which is the uncertainty of waiting time in hospitals.

Through this project, an end-to-end workflow has been implemented, starting from data storage using an SQLite database, followed by data preprocessing, model training using Linear Regression, and evaluation using multiple performance metrics such as R^2 score, Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). The integration of these components into a user-friendly Streamlit application makes the system practical and easy to use.

The results sh'w that factors like patient priority, department, and queue position significantly influence wait time. Emergency cases are handled instantly, while normal cases follow a queue-based estimation, which reflects real-world hospital operations. The analytical visualizations further help in understanding department-wise and priority-wise variations in wait time.

From a business perspective, this system can improve hospital efficiency, reduce patient frustration, and enhance overall service quality by providing transparency in waiting times. It also helps hospital administrators in better resource planning and decision-making.

In conclusion, the project not only fulfills all the requirements of a comprehensive data science capstone but also provides a scalable solution that can be extended further.

Future Improvements:

Integration with real-time hospital data

Use of advanced machine learning models for higher accuracy

Deployment as a web-based or mobile application

Adding doctor availability prediction and scheduling optimization